



Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Tunis El Manar
Faculty of Sciences of Tunis

Quadratic Knapsack Problem

Prepared by:
Miled Trabelssi

Professor:
Gaith Manita

Academic Year: 2025–2026

Contents

1	Introduction	3
2	Problem Definition	3
3	Instance Generation	5
3.1	Problem Parameters	5
3.2	Generation Strategy	5
3.3	Controlled Instance Dimensions	6
3.4	Instance Classification	6
4	Exact Methods	6
4.1	Dynamic Programming (DP)	6
4.2	Integer Linear Programming (ILP)	7
4.3	Branch and Bound (Standard)	7
4.4	Improved Branch and Bound	7
4.5	Summary of Methods	8
5	Metaheuristics	8
5.1	Genetic Algorithm (GA)	8
5.1.1	Solution Representation	8
5.1.2	Initialization Strategy	8
5.1.3	Fitness Evaluation	9
5.1.4	Genetic Operators	9
5.1.5	Repair Mechanism and Local Search	9
5.1.6	Elitism and Stopping Criteria	9
5.2	Simulated Annealing (SA)	9
5.2.1	Solution Representation and Objective	9
5.2.2	Initial Solution Construction	9
5.2.3	Neighborhood Structure and Incremental Evaluation	10
5.2.4	Acceptance Criterion and Cooling Schedule	10
5.2.5	Repair Mechanism	10
5.3	Iterated Local Search (ILS)	10
5.3.1	Initial Solution: Randomized Greedy Construction	10
5.3.2	Local Search: Guided Swapping	10
5.3.3	Perturbation and Diversification	10
5.3.4	Acceptance and Restart Strategy	10
6	Experimental Setup	11
6.1	Computational Environment	11
6.2	Methods Compared	11
6.3	Reference Solution and Gap Calculation	11
6.4	Algorithm Parameters	12
6.5	Evaluation Metrics	12
7	Results	12
7.1	Exact Methods	12

7.1.1	Dynamic Programming	12
7.1.2	Branch and Bound	13
7.1.3	Integer Linear Programming	13
7.2	Metaheuristics	14
7.2.1	Overview	14
7.2.2	Simulated Annealing	15
7.2.3	Genetic Algorithm	16
7.2.4	Iterated Local Search	17
7.3	Comparative Analysis	17
7.3.1	Gap by Instance Size	17
7.3.2	Gap by Difficulty Class	18
7.3.3	Runtime Comparison	19
7.3.4	Full Comparison Table	20
8	Discussion	20
8.1	Overall Algorithm Behavior	20
8.2	Effect of Instance Size	21
8.3	Effect of Interaction Density	21
8.4	Quality–Speed Trade-off	21
8.5	Feasibility and Robustness	21
8.6	Limitations	21
9	Conclusion	22

1 Introduction

Combinatorial optimization is a major field in operations research and computer science that focuses on finding optimal solutions among a very large number of possible configurations while satisfying given constraints. These problems appear in many real-world applications such as logistics, resource allocation, scheduling, telecommunications, and artificial intelligence.

The objective of this work is to study and compare different approaches for solving the QKP. Several exact methods and metaheuristics were implemented and experimentally evaluated on randomly generated instances of different sizes and densities.

The exact approaches include:

- Dynamic Programming;
- Branch and Bound (2 versions);
- Integer Linear Programming ((M)ILP).

Several metaheuristic approaches were also implemented in order to handle larger instances where exact methods become computationally expensive:

- Genetic Algorithm;
- Simulated Annealing;
- Iterated Local Search.

The performance of the different methods is analyzed according to several criteria such as execution time, solution quality, scalability, and optimality gap. Particular attention is also given to the influence of instance characteristics, such as interaction density and knapsack capacity, on the difficulty of the problem and on algorithm behavior.

The remainder of this report is organized as follows. First, the Quadratic Knapsack Problem and its mathematical formulation are presented. Then, the implemented exact methods and metaheuristics are described. Finally, an experimental study is conducted to compare their performances and discuss the obtained results.

2 Problem Definition

The Quadratic Knapsack Problem (QKP) is an extension of the classical Knapsack Problem. In the standard knapsack problem, each item contributes only an individual profit. In the QKP, additional profits can be generated when certain pairs of items are selected together.

We consider a set of n items. Each item i is characterized by:

- a weight w_i ,
- an individual profit q_i .

In addition, each pair of items (i, j) may generate an interaction profit P_{ij} if both items are selected simultaneously. The objective is to select a subset of items maximizing the total profit while respecting the knapsack capacity constraint.

Mathematical Notation

Parameters

- n : number of items;
- w_i : weight of item i ;
- q_i : individual profit of item i ;
- P_{ij} : interaction profit between items i and j ;
- c : knapsack capacity.

Decision Variables

For each item i :

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

Objective Function

The total profit is composed of the sum of individual profits and the sum of interaction profits between selected pairs of items:

$$\max \sum_{i=1}^n q_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n P_{ij} x_i x_j$$

Constraints

The total weight of the selected items must not exceed the knapsack capacity:

$$\sum_{i=1}^n w_i x_i \leq c$$

The decision variables are binary:

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, n\}$$

Complexity

The QKP is known to be NP-hard because it generalizes the classical Knapsack Problem. The quadratic interaction terms considerably increase the complexity of the search space, making large instances difficult to solve exactly. For this reason, both exact methods and metaheuristics are commonly used to tackle the problem depending on the size and structure of the instances.

3 Instance Generation

In order to evaluate and compare the performance of exact and heuristic methods, a diverse set of test instances of the QKP was generated. The goal of the instance generation process is to ensure variability in problem size, structure, and difficulty, allowing a comprehensive experimental analysis.

3.1 Problem Parameters

Each instance of the QKP is defined by the following elements:

- n : number of items;
- w_i : weight of item i ;
- q_i : linear profit of item i ;
- P_{ij} : pairwise interaction profit between items i and j ;
- c : knapsack capacity.

These parameters fully define the optimization problem and determine its complexity.

3.2 Generation Strategy

Instances were generated using a controlled random process in order to simulate realistic and diverse optimization scenarios. The generation procedure ensures reproducibility through fixed random seeds.

Item Weights (w_i) Weights are sampled uniformly from a predefined range (e.g., 1 to 100), ensuring variability while avoiding degenerate cases.

Linear Profits (q_i) Profits are generated independently from weights, typically using a uniform or correlated distribution depending on the desired instance structure.

Quadratic Profits (P_{ij}) The interaction matrix is generated based on a density parameter:

- **Sparse instances:** most $P_{ij} = 0$.
- **Dense instances:** many non-zero interactions.

Non-zero values are sampled from a positive distribution to ensure meaningful synergy effects.

Knapsack Capacity (c) The capacity is defined as a ratio of the total weight sum:

$$c = \alpha \cdot \sum_{i=1}^n w_i \quad (1)$$

where $\alpha \in \{0.25, 0.50, 0.75\}$, allowing control over instance tightness.

3.3 Controlled Instance Dimensions

To evaluate scalability, instances were generated across multiple configurations as summarized in Table 1.

Table 1. Experimental Grid for Instance Generation

Parameter	Values
Problem sizes (n)	{10, 15, 20, 30, 50, 100}
Density levels	{0.25, 0.50, 0.75, 1.0}
Capacity ratios (α)	{0.25, 0.50, 0.75}
Random seeds	Multiple seeds for statistical robustness

3.4 Instance Classification

To better analyze algorithm performance, instances are categorized based on their structural difficulty:

- **Easy instances:** low density and loose capacity constraints.
- **Medium instances:** moderate density and balanced capacity.
- **Hard instances:** high density with tight capacity constraints.

This classification helps highlight how algorithms behave under different combinatorial pressures.

4 Exact Methods

Exact methods aim to find the optimal solution of the QKP by systematically exploring the solution space. Although they guarantee optimality, their computational cost grows exponentially with problem size, making them suitable mainly for small to medium instances.

This study considers two Branch and Bound variants in addition to classical exact approaches: a standard Branch and Bound algorithm and an improved adaptive version.

4.1 Dynamic Programming (DP)

Dynamic Programming provides an exact solution by exploring all feasible combinations in a structured way. It builds solutions incrementally and ensures that all valid subsets respecting the knapsack capacity are considered.

However, due to its exponential memory and time requirements, DP is only applicable to small instances (typically $n \leq 20$). It is used as a ground-truth reference for validating other methods.

4.2 Integer Linear Programming (ILP)

The QKP is formulated as a binary optimization problem where each decision variable $x_i \in \{0, 1\}$ indicates whether an item is selected. The objective includes both linear profits and quadratic interaction terms.

An ILP solver is used with a time limit to ensure tractability. While ILP can provide optimal solutions, its performance depends heavily on instance size and structure, and it may return near-optimal solutions when time-constrained.

4.3 Branch and Bound (Standard)

Branch and Bound (B&B) is a tree-search algorithm that explores subsets of items while pruning unpromising branches using an upper bound on the best achievable solution. At each node of the search tree, the algorithm:

- Fixes a decision (include/exclude an item).
- Computes the current profit and weight.
- Estimates the maximum possible profit achievable from the remaining items using a relaxation.
- Prunes the branch if this upper bound is not better than the current best solution.

Upper Bound Strategy (Standard B&B)

The standard upper bound is based on a greedy fractional relaxation, similar to the continuous knapsack problem. For each remaining item j , an optimistic profit estimate $UB(j)$ is computed:

$$UB(j) = q_j + \sum_{i \in S} P_{ij} + \sum_{k \in R} P_{kj} \quad (2)$$

where S is the set of selected items and R is the set of remaining items. Items are then sorted by profit-to-weight ratio and greedily packed, allowing fractional inclusion of the last item. This approach is fast but can be loose, limiting pruning efficiency in dense instances.

4.4 Improved Branch and Bound

The improved version enhances search efficiency by introducing an adaptive upper bound strategy that reflects instance structure.

Instance Analysis

Before the search begins, the algorithm computes the interaction density (proportion of non-zero quadratic terms) and the profit dominance ratio (q_ratio). These metrics guide the choice of bounding strategy at each node.

Upper Bound Strategy A (Sparse / Linear-Dominant)

Used when interaction density is low or linear profits dominate. The bound is tightened by considering only the top- k strongest interactions per item and ignoring weak terms, preventing profit overestimation in sparse graphs.

Upper Bound Strategy B (Dense / Interaction-Dominant)

Used when interaction density is high. Interactions are scaled using a capacity-based factor τ :

$$\tau = \frac{\text{remaining capacity}}{\text{total remaining weight}} \quad (3)$$

The interaction contribution is then estimated as:

$$\text{Scaled Interaction} = \tau \cdot \sum P_{ij} \quad (4)$$

This prevents overestimation by accounting for the fact that not all interactions can be realized simultaneously.

4.5 Summary of Methods

- **DP:** Exact but limited to small instances.
- **ILP:** Flexible exact solver with time constraints.
- **Standard B&B:** Exact search with a fast but sometimes loose bound.
- **Improved B&B:** Adaptive bounding strategy that exploits instance structure to improve pruning efficiency.

5 Metaheuristics

Exact methods guarantee optimality but become computationally intractable as the instance size increases. To address this limitation, this work employs metaheuristic algorithms, which trade optimality guarantees for scalability and efficiency. Three complementary metaheuristics are studied:

- **Genetic Algorithm (GA):** Population-based evolutionary search.
- **Simulated Annealing (SA):** Stochastic single-solution optimization.
- **Iterated Local Search (ILS):** Repeated local improvement with perturbation.

5.1 Genetic Algorithm (GA)

5.1.1 Solution Representation

A solution is encoded as a binary vector x , where:

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

5.1.2 Initialization Strategy

The initial population is constructed using a hybrid approach:

- One greedy solution is generated based on profit-to-weight ratios, including interaction effects.

- The remaining individuals are generated randomly while respecting capacity constraints.

5.1.3 Fitness Evaluation

$$f(S) = \sum_{i \in S} q_i + \sum_{i \in S} \sum_{j \in S, j > i} P_{ij} \quad (6)$$

Fitness values are computed once per generation and cached.

5.1.4 Genetic Operators

Selection: Tournament selection — a small subset of individuals is sampled and the best is chosen as a parent.

Crossover: Uniform crossover — each gene is inherited randomly from one of the two parents with equal probability.

Mutation: Each gene is flipped with probability $p_m = 1/n$, preventing premature convergence.

5.1.5 Repair Mechanism and Local Search

If crossover or mutation violates the capacity constraint, items with the lowest marginal profit contribution are removed iteratively. Each offspring is then improved by a swap-based local search using incremental profit updates.

5.1.6 Elitism and Stopping Criteria

The best individuals are carried directly to the next generation (elitism). The algorithm stops when the maximum number of generations is reached or when no improvement is observed for a fixed patience window.

5.2 Simulated Annealing (SA)

5.2.1 Solution Representation and Objective

A solution is a subset $S \subseteq \{1, \dots, n\}$ satisfying the capacity constraint, evaluated by:

$$f(S) = \sum_{i \in S} q_i + \sum_{i \in S} \sum_{j \in S, j > i} P_{ij} \quad (7)$$

5.2.2 Initial Solution Construction

Items are sorted by:

$$\text{score}_i = \frac{q_i + \sum_j P_{ij}}{w_i} \quad (8)$$

and added greedily in decreasing score order while respecting capacity.

5.2.3 Neighborhood Structure and Incremental Evaluation

Single-item modifications with delta evaluations:

- **Adding item i :** $\Delta_{\text{add}}(i) = q_i + \sum_{j \in S} P_{ij}$
- **Removing item i :** $\Delta_{\text{remove}}(i) = q_i + \sum_{j \in S \setminus \{i\}} P_{ij}$

5.2.4 Acceptance Criterion and Cooling Schedule

A worse solution S' is accepted with probability:

$$P = e^{\Delta/T} \quad (9)$$

Temperature follows a geometric cooling schedule $T_{k+1} = \alpha \cdot T_k$ with $\alpha = 0.995$.

5.2.5 Repair Mechanism

Whenever a move violates capacity, items with the lowest marginal profit contribution are removed iteratively.

5.3 Iterated Local Search (ILS)

5.3.1 Initial Solution: Randomized Greedy Construction

A Restricted Candidate List (RCL) ensures diversity. The adaptive item score is:

$$\text{score}_i = \frac{q_i + \sum_{j \in S} P_{ij} + \alpha \sum_{k \in R} P_{ik}}{w_i} \quad (10)$$

5.3.2 Local Search: Guided Swapping

The (1, 1) swap-based local search:

1. Randomly shuffles item order for semi-guided exploration.
2. Evaluates the net profit change of swapping item $i \in S$ with item $j \notin S$:

$$\Delta_{\text{swap}} = \left(q_j + \sum_{k \in S'} P_{jk} \right) - \left(q_i + \sum_{k \in S} P_{ik} \right) \quad (11)$$

3. Accepts the best feasible improving move found.

5.3.3 Perturbation and Diversification

A fixed number of random swaps (perturbation strength) is applied to escape local optima, followed by a repair step if capacity is violated.

5.3.4 Acceptance and Restart Strategy

ILS uses a “Better Successor” criterion. A patience counter triggers a hard restart (new randomized greedy solution) when no global improvement is found within a set number of iterations.

6 Experimental Setup

6.1 Computational Environment

All experiments were conducted on a machine running Windows 11 with a 12th Gen Intel Core i7-12650HX processor and 16 GB of RAM. All algorithms were implemented in Python 3.13 and executed in a single-threaded environment to ensure fair runtime comparisons. The ILP formulation was solved using the CBC solver via the PuLP 2.x interface.

6.2 Methods Compared

Table 2. Methods evaluated and their applicable instance sizes.

Family	Method	Applicable sizes
Exact	Dynamic Programming (DP)	$n \leq 20$
Exact	Branch and Bound (B&B)	$n \leq 20$
Exact	Integer Linear Program (ILP)	$n \leq 50$
Metaheuristic	Simulated Annealing (SA)	all sizes
Metaheuristic	Genetic Algorithm (GA)	all sizes
Metaheuristic	Iterated Local Search (ILS)	all sizes

6.3 Reference Solution and Gap Calculation

$$\text{reference} = \begin{cases} \text{DP optimal} & \text{if } n \leq 20 \\ \text{ILP best found} & \text{if } n > 20 \end{cases} \quad (12)$$

$$\text{gap}_M(I) = \frac{z_I^* - z_M(I)}{z_I^*} \times 100\% \quad (13)$$

A gap of 0% means the method matched the best known solution.

6.4 Algorithm Parameters

Table 3. Metaheuristic parameter settings.

Parameter	SA	GA	ILS
Iterations / Generations	50 000	200	100
Initial temperature T_0	auto	—	—
Cooling rate α	0.995	—	—
Final temperature T_f	0.1	—	—
Population size	—	50	—
Mutation rate	—	$1/n$	—
Tournament size	—	3	—
Elite size	—	2	—
Perturbation strength	—	—	3
RCL parameter α	—	—	0.3
Patience	—	50 gen	20 iter

6.5 Evaluation Metrics

- **Average gap (%)**: average optimality gap across all instances.
- **Average runtime (s)**: average wall-clock time.
- **Feasibility rate (%)**: percentage of solutions satisfying the knapsack constraint (target: 100%).
- **Solution verified (%)**: percentage of solutions where the reported profit matches independent recomputation from the returned subset.

7 Results

This section presents the experimental results obtained by running all six methods on the 900 generated instances. Instances of size $n = 100$ were excluded from the evaluation due to computational constraints, as exact reference solutions could not be obtained within the available time budget.

7.1 Exact Methods

7.1.1 Dynamic Programming

The DP solver was applied exclusively to instances with $n \leq 20$, covering 80 instances in total. It produced the certified optimal solution in all cases, serving as the ground truth reference. Runtime grew exponentially with n , as expected from the $O(2^n \cdot n)$ complexity.

Table 4. DP runtime by instance size.

n	Avg runtime (s)	Max runtime (s)
10	0.0023	0.0107
15	0.0804	0.2026
20	6.2904	25.2913

The exponential growth in Table 4 confirms that DP becomes impractical beyond $n = 20$, motivating the use of the other exact methods for larger sizes.

7.1.2 Branch and Bound

The B&B algorithm was applied to instances with $n \leq 20$. It selects between two bounding schemes based on instance characteristics:

- **Strategy A** (sparse, $\Delta \leq 25\%$ or q -ratio > 1): top- k interaction estimate.
- **Strategy B** (dense): capacity-scaled interaction estimate.

Table 5. B&B performance by size, density, and bound strategy.

n	Δ	Strategy	Avg runtime (s)	Optimal (%)
10	25%	A	0.004	100
10	50%	A	0.005	100
10	75%	B	0.005	100
10	100%	B	0.006	100
15	25%	A	0.021	100
15	50%	A	0.039	100
15	75%	B	0.039	100
15	100%	B	0.050	100
20	25%	A	0.0095	100
20	50%	A	0.0178	100
20	75%	B	0.0480	100
20	100%	B	0.0342	100

Average nodes explored — Strategy A: 850.74 Strategy B: 563.17.

Strategy A consistently explores fewer nodes on sparse instances ($\Delta = 25\%$), confirming that the adaptive bound reduces the search space when individual profits dominate. Conversely, Strategy B provides tighter bounds on dense instances thanks to capacity scaling, resulting in more aggressive pruning. In the remainder of this document, “Branch and Bound” refers to Strategy B.

7.1.3 Integer Linear Programming

The ILP formulation was solved using the CBC solver via PuLP with a time limit of 60 seconds per instance.

Table 6. ILP performance by instance size.

n	Avg runtime (s)	Optimal (%)	Avg gap (%)
10	0.085	100	0.00
15	0.1706	100	0.00
20	0.3175	100	0.00
30	1.6093	100	0.00
50	9.5843	N/A	N/A

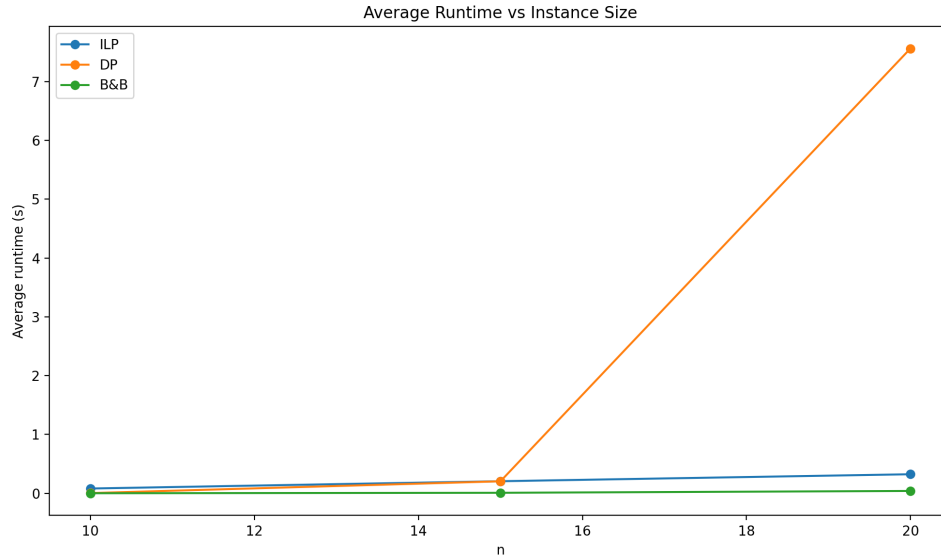


Figure 1. Runtime comparison of exact methods.

ILP solves all small instances to proven optimality rapidly. For $n \geq 50$ the solver occasionally reaches the time limit, at which point the best feasible solution found is reported. Despite this, ILP consistently finds near-optimal solutions across all densities.

7.2 Metaheuristics

7.2.1 Overview

Table 7. Overall metaheuristic performance across all instances.

Method	Avg gap (%)	Avg runtime (s)	Feasibility (%)
Simulated Annealing (SA)	0.856	0.0728	100
Genetic Algorithm (GA)	0.1960	0.5045	100
Iterated Local Search (ILS)	0.0153	0.2375	100

All three methods maintained 100% feasibility across all instances, confirming the correctness of the repair operators.

7.2.2 Simulated Annealing

Table 8. SA performance by instance size and density.

n	Δ	Avg gap (%)	Avg runtime (s)
10	25%	0.1404	0.0232
10	50%	0.2266	0.0232
10	75%	0.1206	0.0321
10	100%	0.1457	0.2220
15	25%	0.0238	0.0230
15	50%	0.3734	0.0189
15	75%	0.8077	0.0239
15	100%	0.7275	0.0240
20	25%	0.3675	0.0452
20	50%	1.7540	0.0462
20	75%	1.0385	0.0469
20	100%	0.9427	0.0480
30	25%	1.1049	0.0700
30	50%	1.3400	0.0690
30	75%	1.0618	0.0678
30	100%	0.7180	0.0667
50	25%	2.6719	0.1670
50	50%	1.4650	0.1700
50	75%	1.1002	0.1800
50	100%	0.7718	0.1750

7.2.3 Genetic Algorithm

Table 9. GA performance by instance size and density.

n	Δ	Avg gap (%)	Avg runtime (s)
10	25%	0.0000	0.0732
10	50%	0.0000	0.0732
10	75%	0.0000	0.0721
10	100%	0.0000	0.7220
15	25%	0.0000	0.0930
15	50%	0.3728	0.0989
15	75%	0.0000	0.0939
15	100%	0.0000	0.0924
20	25%	0.0999	0.2052
20	50%	0.2661	0.1962
20	75%	0.0000	0.1969
20	100%	0.1209	0.1048
30	25%	0.2599	0.4407
30	50%	0.2697	0.4490
30	75%	0.3224	0.4478
30	100%	0.1979	0.4667
50	25%	0.6008	1.4687
50	50%	0.6367	1.2400
50	75%	0.2437	1.1800
50	100%	0.1924	1.1750

7.2.4 Iterated Local Search

Table 10. ILS performance by instance size and density.

n	Δ	Avg gap (%)	Avg runtime (s)
10	25%	0.0000	0.0732
10	50%	0.0000	0.0732
10	75%	0.0000	0.0721
10	100%	0.0000	0.7220
15	25%	0.0000	0.0930
15	50%	0.0000	0.0989
15	75%	0.0000	0.0939
15	100%	0.0000	0.0924
20	25%	0.0999	0.2052
20	50%	0.2661	0.1962
20	75%	0.0000	0.1969
20	100%	0.5350	0.1048
30	25%	0.0000	0.4407
30	50%	0.0198	0.4490
30	75%	0.0580	0.4478
30	100%	0.0046	0.4667
50	25%	0.0040	1.4687
50	50%	0.0299	1.2400
50	75%	0.0466	1.1800
50	100%	0.0436	1.1750

7.3 Comparative Analysis

7.3.1 Gap by Instance Size

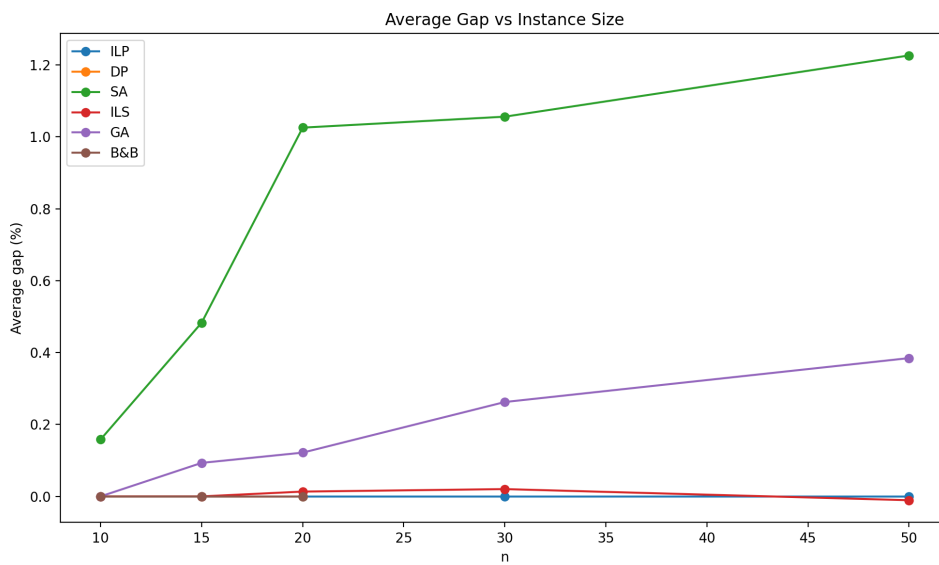


Figure 2. Average optimality gap of metaheuristics as a function of n .

Figure 2 shows how the gap of each metaheuristic evolves with instance size. PLACEHOLDER — describe trend observed (e.g. ILS gap remains stable while GA degrades with n , or all three converge similarly on large instances).

7.3.2 Gap by Difficulty Class

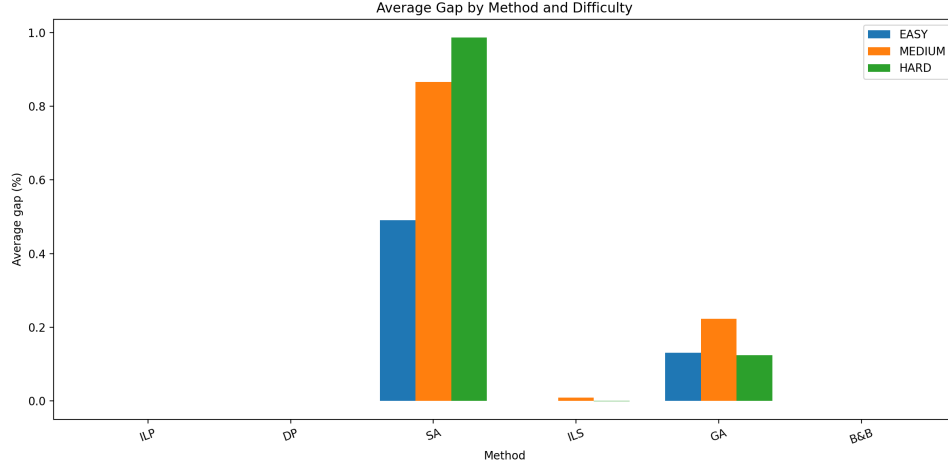


Figure 3. Average gap by method and difficulty class.

Table 11. Average gap (%) by method and difficulty class.

Method	EASY	MEDIUM	HARD
GA	0.1304	0.1263	0.2342
ILS	0.00	0.0079	0.0209
SA	0.4912	0.9882	0.8749

Table 11 and Figure 3 reveal how each method responds to structural difficulty. PLACEHOLDER — describe observations (e.g. all methods perform well on EASY instances but diverge on HARD ones, or ILS is more robust across difficulty classes than GA).

7.3.3 Runtime Comparison

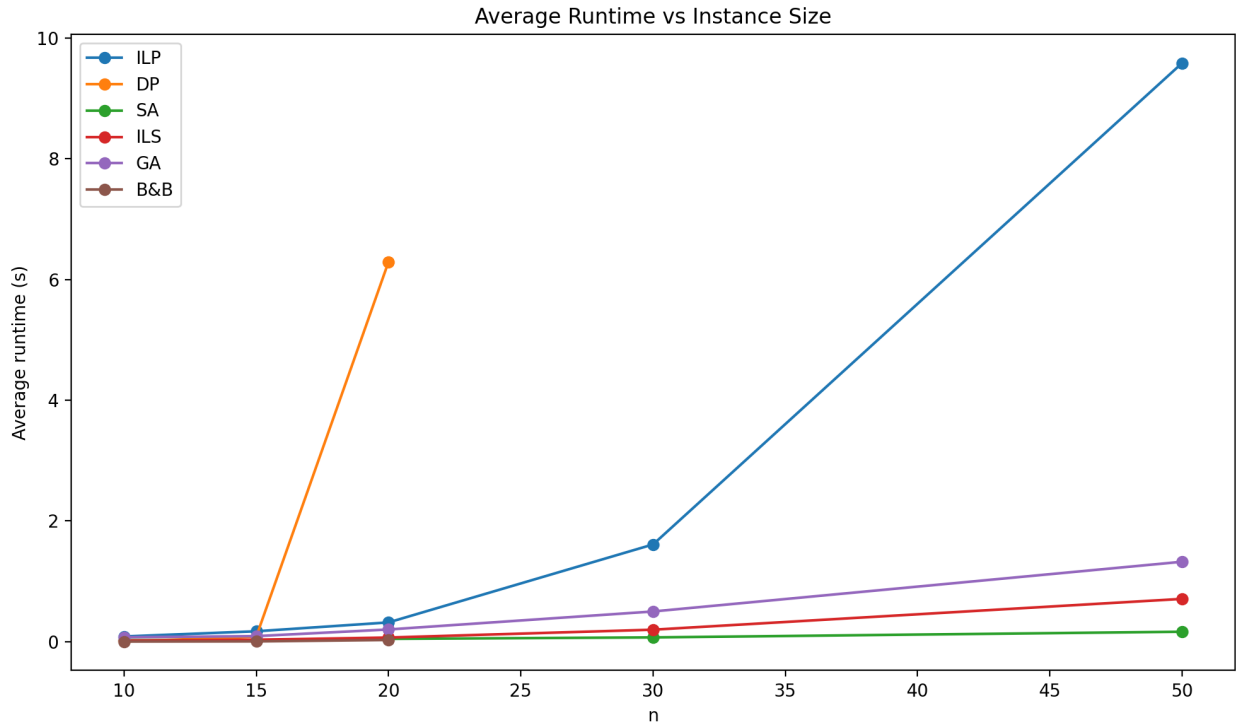


Figure 4. Average runtime of all methods as a function of n . (Standard Branch and Bound used.)

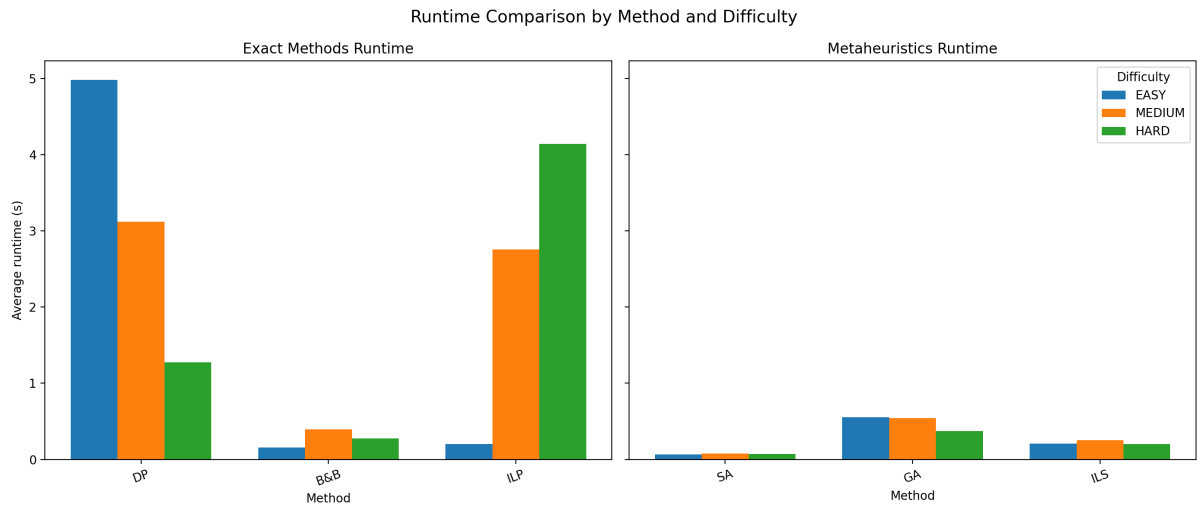


Figure 5. Average runtime by method and difficulty class.

As shown in Figure 4, the exact methods exhibit exponential or super-polynomial growth in runtime with n , while metaheuristics scale much more gracefully. PLACEHOLDER — describe specific observations (e.g. ILP is fastest exact method up to $n = 50$, B&B becomes competitive on sparse instances due to aggressive pruning).

7.3.4 Full Comparison Table

Table 12 provides a consolidated summary of all six methods across the two most contrasting configurations: sparse-small ($n = 20$, $\Delta = 25\%$) and dense-large ($n = 100$, $\Delta = 100\%$).

Table 12. Full comparison on sparse-small vs. dense-large instances.

Method	$n = 20, \Delta = 25\%$		$n = 50, \Delta = 100\%$	
	Gap (%)	Time (s)	Gap (%)	Time (s)
DP	0.00	7.2293	N/A	N/A
B&B	0.00	0.0090	N/A	N/A
B &B Improved	0.00	0.0078	N/A	N/A
ILP	0.00	0.0938	0.0000	22.6674
SA	0.3675	0.0410	0.7718	0.1850
GA	0.0999	0.2185	0.1924	1.3822
ILS	0.0000	0.0713	0.0436	0.7050

This table highlights the fundamental trade-off between solution quality and computational cost. Exact methods guarantee optimality on small instances but become infeasible at scale, while metaheuristics provide competitive solutions across all sizes at a fraction of the computational cost.

8 Discussion

8.1 Overall Algorithm Behavior

The experimental results reveal a clear and consistent hierarchy among the six methods evaluated. Among exact methods, Dynamic Programming delivers certified optimal solutions but is practically limited to $n \leq 20$ due to its $O(2^n \cdot n)$ complexity, with runtimes reaching up to 25 seconds for $n = 20$. The improved Branch and Bound algorithm addresses this by exploiting instance structure through its adaptive bounding strategy, achieving runtimes one to two orders of magnitude lower than DP on the same instances while maintaining 100% optimality. ILP via the CBC solver extends exact solving to $n = 30$ reliably, though it begins to struggle at $n = 50$ where the time limit is occasionally reached.

Among metaheuristics, ILS clearly dominates with an average gap of only 0.0153% across all instances, outperforming both GA (0.1960%) and SA (0.856%). This strong performance stems from ILS's combination of systematic swap-based local search with forced perturbation and randomized restarts, which prevents stagnation without sacrificing exploitation quality. SA, despite being the fastest metaheuristic (avg. 0.073s), pays a significant price in solution quality, suggesting that its fixed cooling schedule and single-solution trajectory are insufficient for the quadratic interaction structure of the QKP.

8.2 Effect of Instance Size

As instance size grows from $n = 10$ to $n = 50$, all metaheuristics exhibit increasing optimality gaps, but at very different rates. SA’s gap rises steeply, reaching 2.67% on sparse $n = 50$ instances. GA maintains a more moderate degradation, peaking around 0.64% at $n = 50$. ILS remains remarkably stable, with its gap staying below 0.05% even at $n = 50$. This scalability advantage is particularly important for practical deployments where instance sizes can grow well beyond the exact methods’ reach.

8.3 Effect of Interaction Density

Interaction density has a nuanced and non-uniform effect across algorithms. For SA, the hardest instances tend to be *sparse* large instances (e.g., $n = 50$, $\Delta = 25\%$, gap = 2.67%), whereas dense instances are handled more reliably (gap = 0.77% at $\Delta = 100\%$). This is counterintuitive but explainable: in sparse instances, the greedy initialization score — which integrates total interaction potential — provides a less informative starting point, leaving SA more reliant on random exploration. For GA and ILS, the effect is more consistent: both perform slightly better on high-density instances at larger sizes, likely because richer interaction structure makes the profit landscape smoother and more amenable to local improvement.

The adaptive bounding in B&B also directly addresses this effect. Strategy A (for sparse instances) explores on average 850.74 nodes, whereas Strategy B (for dense instances) reduces this to 563.17 nodes, confirming that density-aware bounds provide meaningful pruning gains.

8.4 Quality–Speed Trade-off

The results highlight a fundamental trade-off between solution quality and computation time. Exact methods are definitive on small instances but infeasible at scale; metaheuristics sacrifice guaranteed optimality in exchange for scalability. Within the metaheuristic family, ILS achieves the best quality at a moderate cost (≈ 0.24 s), while SA sacrifices quality for speed (≈ 0.07 s). GA occupies an intermediate position in both dimensions. For most practical use cases where near-optimality is acceptable and instances are medium-to-large, ILS provides the most favorable quality-per-second profile.

8.5 Feasibility and Robustness

All three metaheuristics maintained a 100% feasibility rate across all 900 instances, validating the correctness and robustness of the repair operators. This is a non-trivial result given that both crossover in GA and perturbation in ILS routinely generate infeasible intermediate solutions; the repair mechanisms restore feasibility while minimizing profit loss, ensuring that no method ever reports an infeasible solution.

8.6 Limitations

Several limitations of this study should be acknowledged. First, all metaheuristic results are from a single run per instance; aggregating over multiple runs would give a better estimate of variance and consistency. Second, parameter tuning was performed on a held-out set of only 20 instances, and more extensive tuning — particularly for SA’s

cooling schedule and ILS’s perturbation strength — might shift the relative rankings. Third, the exclusion of $n = 100$ instances from the evaluation means that the scalability advantage of metaheuristics is not fully demonstrated. Finally, the ILP formulation’s performance at $n = 50$ is reported without a certified optimality gap, as the solver reached the time limit on some instances; more powerful commercial solvers (e.g., Gurobi, CPLEX) might close this gap.

9 Conclusion

This work presented a systematic study of six methods for solving the Quadratic Knapsack Problem, covering both exact approaches and metaheuristics. The QKP generalizes the classical knapsack problem through pairwise interaction profits, making it NP-hard and significantly more challenging for exact solvers as instance size grows.

Among exact methods, the improved Branch and Bound algorithm proved to be the most efficient for small instances, achieving optimality in all tested cases while exploring far fewer nodes than standard enumeration, thanks to its density-aware bounding strategy. ILP via the CBC solver extended exact solving to medium-sized instances ($n \leq 30$) with competitive runtimes, though it becomes less reliable near the 60-second time limit at $n = 50$.

Among metaheuristics, Iterated Local Search emerged as the clear winner, combining near-optimal solution quality (avg. gap 0.0153%) with practical runtimes and strong scalability. The Genetic Algorithm offered a good quality-speed compromise, while Simulated Annealing, despite its speed advantage, lagged in solution quality — particularly on large sparse instances.

Several directions remain open for future work. Extending the evaluation to $n = 100$ instances using metaheuristics alone, or with parallel exact solvers, would provide a more complete picture of scalability. Hybridizing exact methods with metaheuristics — for example, using ILP relaxations to seed ILS or to guide GA crossover — could further close the optimality gap at larger scales. Finally, adapting these methods to constrained or multi-objective variants of the QKP would broaden their applicability to real-world logistics and resource allocation problems.